HashiCorp Certified: Terraform Associate（004）　目標 5a～5d　#11

# モジュール

ルートと子、module ブロックと source（ローカル・Registry・プライベート・Git と ref・サブディレクトリ）、入力と出力とスコープ、モジュールの中のプロバイダー、平らな構成、version と init -upgrade、標準モジュール構造と公開の条件

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の Modules overview・Creating Modules・module block reference・Module Sources・Use modules in your configuration・Providers Within Modules・Module Composition・Standard Module Structure・Version constraints、Terraform Registry の Publish modules）を書いた。英語の原文は『』で引用した

第1章

# しくみとソース

どこからコードを取ってくるか。

# ルートモジュールと子モジュール

**ルートモジュール**
plan・apply を実行するディレクトリの .tf

↓ module ブロックで呼ぶ

**子モジュール**
何度でも呼べる。同じ source を別のラベルで

↓

**入れ子のモジュール**
子がさらに呼ぶ（深くしすぎない）

子のリソースもルートと同じステートで管理する。

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"          # Registry だけ
  name    = "main"            # モジュール固有の入力
  cidr    = "10.0.0.0/16"
  # count / for_each（どちらか）・providers・depends_on
}
resource "aws_security_group" "web" {
  vpc_id = module.vpc.vpc_id  # 出力を参照
}
```

source はリテラルの文字列だけ。変えたら terraform init（コードは .terraform/modules へ）。

出典：Modules overview／module block reference／Use modules in your configuration

# source の書き方

| ソース | 例 | 覚えどころ |
|---|---|---|
| ローカルパス | "./consul"・"../modules/vpc" | ./ か ../ で始める。インストールせずそのまま。version 不可 |
| 公開 Registry | "hashicorp/consul/aws" | <NAMESPACE>/<NAME>/<PROVIDER>。version 可 |
| プライベート Registry | "app.terraform.io/example-corp/k8s-cluster/azurerm" | 先頭にホスト名（localterraform.com も可）。トークンで認証 |
| GitHub | "github.com/hashicorp/example" | Git の別名。HTTPS でクローン |
| 一般の Git | "git::https://example.com/vpc.git?ref=v1.2.0" | ref なしは既定のブランチ（HEAD）。?depth=1 は浅いクローン |
| HTTP・S3・GCS | "https://example.com/vpc-module.zip" | アーカイブを取ってくる |
| サブディレクトリ | "git::https://example.com/network.git//modules/vpc?ref=v1.2.0" | // のあとがパッケージの中。クエリは最後 |

> すすめ：くり返しをまとめるだけならローカルパス、多くの構成で共有するならレジストリ。/ で始まる絶対パスはすすめない。

出典：Module Sources／module block reference

第2章

# 値の受け渡しとスコープ

入力と出力だけで外とつながる。

# 入力・出力・カプセル化

```
# 親（ルート）
module "web" {
  source  = "./modules/web"
  servers = 3                    # 親 → 子
}
output "web_ip" {
  value = module.web.instance_ip   # 子 → 親
}

# 子（modules/web）
variable "servers" { type = number }   # default なし＝必須
resource "aws_instance" "this" {
  count = var.servers
}
output "instance_ip" {
  value = aws_instance.this[*].public_ip
}
```

**variable ＝ 引数**

**locals・リソース ＝ 中だけ**

**output ＝ 戻り値**

子は親の var.・local.・リソースを参照できない。親も module.web.aws_instance.this は参照できない。

TF_VAR_・tfvars はルートの変数用。子へは module ブロックの引数で。

for_each なら module.web["a"].instance_ip、count なら module.web[0].instance_ip。

出典：Creating Modules／Use modules in your configuration／Output values

# モジュールの中のプロバイダーと構成

```
# ルートだけに provider ブロック
provider "aws" { region = "us-east-1" }
provider "aws" {
  alias  = "west"
  region = "us-west-2"
}
module "web_west" {
  source    = "./modules/web"
  providers = { aws = aws.west }   # 明示して渡す
}
```

渡さなければ既定の設定が暗黙に継承される。

呼ばれるモジュールに provider ブロックを書かない（count・for_each・depends_on と両立しない）。required_providers は各モジュールで宣言。

**平らな構成（composition）**
子は 1 段。ルートで出力 → 入力をつなぐ

**深い入れ子**
使いまわしにくい

**薄い包み（1 リソースだけ）**
すすめない。そのリソースを直接使う

モジュールの中のリソースの作り直し：terraform apply -replace=module.example.aws_instance.example

出典：Providers Within Modules／Module Composition／Creating Modules

第3章

# バージョンと公開

固定して、都合のよいときに上げる。

# version と init -upgrade

| 制約 | 使ってよい版 |
|---|---|
| = 1.2.0（演算子なしも） | 1.2.0 だけ |
| >= 1.2.0, < 2.0.0 | 1.2.0 以上 2.0.0 未満 |
| ~> 1.2.0 | 1.2.x（1.3.0 は不可） |
| ~> 1.2 | 1.x（2.0 は不可） |

> version を書かなければ最新版。インストール済みで制約に合う最新を使う。

```
$ terraform init            # 入っているものが合えばそのまま
$ terraform init -upgrade   # 制約が許す最新に上げる
```

| だれが | すすめられる制約 |
|---|---|
| 社外のモジュールを使う | 特定のバージョンを指定 |
| 社内の semver を守るモジュール | 範囲で指定してよい |
| 再利用するモジュールの中 | Terraform・プロバイダーは最低のバージョンだけ（>=） |
| ルートモジュール | プロバイダーは ~> で上限と下限 |

出典：module block reference（version）／Version constraints／Use modules in your configuration

# 標準モジュール構造と公開

```
terraform-aws-consul/      # terraform-<PROVIDER>-<NAME>
├── README.md
├── LICENSE
├── main.tf                # 入口
├── variables.tf           # description を書く
├── outputs.tf
├── modules/               # 入れ子のモジュール
│   ├── consul-cluster/
│   └── consul-security-group/
└── examples/              # 使い方の例
```

必須はルートモジュールだけ。README のある入れ子モジュールは外から使ってよいもの。

| 公開 Registry の条件 |
| --- |
| GitHub の公開リポジトリ |
| 名前は terraform-<PROVIDER>-<NAME> |
| リポジトリの説明（短い説明になる） |
| 標準モジュール構造 |
| x.y.z のタグ（v1.0.4・0.9.2） |

新しい版はタグを push するだけ。プライベートレジストリなら GitHub の公開リポジトリの条件はいらない。

出典：Standard Module Structure／Publish modules

# 本番での問われ方

| ねらい | 見るところ |
| --- | --- |
| source（5a） | ./ と ../、Registry の 3 部、ホスト名、git:: と ?ref=、//、リテラルだけ、変えたら init |
| スコープ（5b） | 親の変数は見えない、引数で渡す、出力で返す、module.名前.出力 |
| 使い方（5c） | providers 引数、provider ブロックはルートだけ、平らな構成、count・for_each、-replace のアドレス |
| バージョン（5d） | version は Registry だけ、~>、既定は最新、init -upgrade、固定のすすめ |

出典：Exam Content List（004）5a〜5d／本資料の問題演習（全 30 問）